TEXORA · LECTURE SHEET

Memory Organization

The memory hierarchy from registers to tape, primary memory in detail — RAM, the ROM family, and cache with locality — secondary storage devices, and the units, addresses, and conversions that describe how much a memory can hold.

CSE-2105 · FUNDAMENTALS OF COMPUTING · TOPIC 2 · LEVEL 2 TERM 1

1. Why memory is organised as a hierarchy

No single memory technology is fast, large, and cheap at the same time. Fast memory costs far more per byte than slow memory, so a computer combines several levels: a small amount of very fast, expensive memory close to the processor, backed by progressively larger, slower, and cheaper levels. This arrangement is called the **memory hierarchy**, and it works because programs do not access all their data evenly — at any moment they work intensively on a small working set. Keeping that working set in the fast levels gives most of the speed of an all-fast memory at a cost close to an all-cheap one.

Four properties change steadily as we move down the hierarchy, away from the CPU: **access time** increases (the memory gets slower), **capacity** increases, **cost per byte** decreases, and the **frequency of access** by the CPU decreases. Understanding any memory question usually means placing a device correctly in this trade-off.

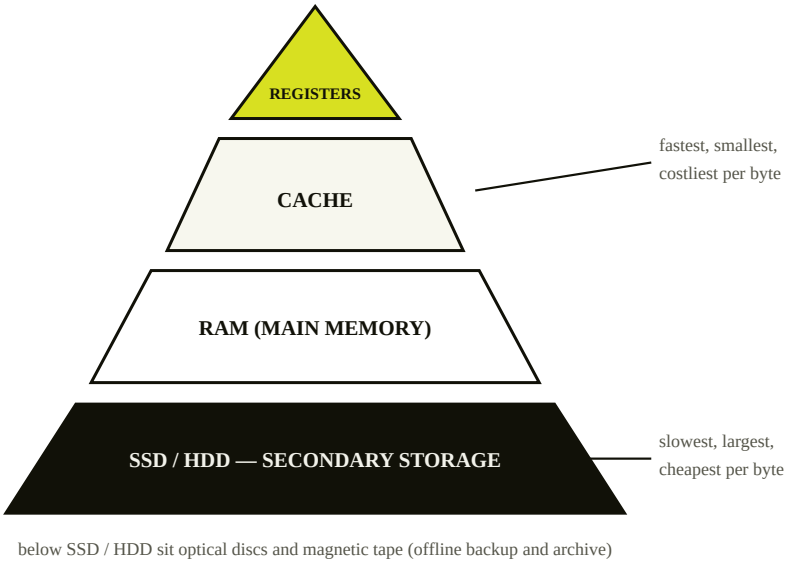


Figure 1 — The memory hierarchy: registers, cache, RAM, then SSD/HDD, with optical and tape below

LEVEL	TYPICAL SIZE	ACCESS TIME (ORDER)	CHARACTER
Registers	Tens to hundreds of bytes	Under 1 ns	Inside the CPU; holds operands of the current instruction
Cache	KB to tens of MB	About 1–10 ns	Very fast SRAM between CPU and RAM
RAM (main memory)	GB	About 50–100 ns	Holds running programs and their data; volatile
SSD	Hundreds of GB to TB	About 50–100 microseconds	Flash-based secondary storage; no moving parts
HDD	TB	Several milliseconds	Magnetic disks; mechanical movement dominates the delay
Optical / tape	GB per disc; TB per tape	Seconds (tape seek)	Removable media for distribution, backup, and archive

The orders of magnitude matter more than the exact figures: each level is roughly a hundred to a thousand times slower than the one above it, and roughly a thousand times larger. That is why a cache miss that forces the CPU to wait for a hard disk feels, in processor time, like waiting days.

2. Primary memory: RAM

Primary memory (main memory) is the storage the CPU can address directly. It holds the program being executed and the data it is working on. Its two great families are RAM and ROM. **Random Access Memory (RAM)** is read-write memory: any location can be reached in the same time as any other, which is what “random access” means. RAM is **volatile** — its contents vanish when the power is switched off — so it is a workspace, never an archive.

FEATURE	SRAM (STATIC RAM)	DRAM (DYNAMIC RAM)
Storage cell	A flip-flop of about six transistors per bit	One transistor and one tiny capacitor per bit
Refreshing	Not needed while power is on; the flip-flop holds its state	Needed many times per second; the capacitor leaks charge away
Speed	Faster	Slower
Density and cost	Fewer bits per chip; expensive per byte	Many bits per chip; cheap per byte

Typical use	Cache memory	Main memory (RAM) of the computer

The comparison explains the hierarchy in miniature: SRAM is used where speed matters most and capacity is small (cache), while DRAM is used where gigabytes are needed at an affordable price (main memory), accepting the cost of refresh circuitry.

3. Primary memory: the ROM family

Read-Only Memory (ROM) is non-volatile memory whose contents are retained without power. Its classic role is to hold the firmware that starts the machine — the boot program the CPU runs before any operating system is loaded from disk. The family members differ in how, and how often, they can be written.

TYPE	HOW IT IS WRITTEN	ERASING AND REUSE
ROM	Programmed once during manufacture (mask-programmed)	Cannot be changed
PROM	Programmable once by the user with a special PROM programmer that burns microscopic fuses	One-time only; a mistake cannot be undone
EPROM	Programmed electrically by the user	Erased as a whole by ultraviolet light through a quartz window on the chip, then reprogrammed; limited erase cycles
EEPROM	Programmed electrically, in place, byte by byte	Erased electrically without removal from the circuit; slower to write and costlier per byte. Flash memory is a block-erasable descendant of EEPROM and is the basis of SSDs and USB drives.

A useful way to remember the family: each step from ROM to EEPROM trades cost and write convenience. Firmware that will never change suits mask ROM; firmware that may need a field update suits EEPROM or flash.

4. Cache memory and locality

Cache memory is a small, very fast SRAM store placed between the CPU and main memory. It holds copies of the instructions and data the CPU is most likely to need next. When the CPU asks for a word, the cache is checked first: a **hit** supplies the word at cache speed, while a **miss** means fetching from

RAM (and usually copying a whole block into the cache for next time). A well-designed cache reaches hit rates above 90 percent, so the average access time stays close to cache speed. Modern processors use several levels — L1 closest and smallest, L2 larger, L3 larger still and shared between cores.

Cache works because of **locality of reference**. **Temporal locality:** a location used recently is likely to be used again soon (a loop counter, a running total). **Spatial locality:** locations near a used location are likely to be used soon (the next instruction in sequence, the next element of an array). Fetching a whole block on a miss exploits spatial locality; keeping the block exploits temporal locality. Without locality, caching would be useless — which is why programs with orderly access patterns run far faster than their instruction counts alone would suggest.

5. Secondary storage

Secondary storage (auxiliary memory) holds data and programs permanently, in bulk, at low cost per byte. It is non-volatile and is not directly addressable by the CPU: data must first be copied into RAM before processing. The delay involved is acceptable because secondary storage is used for keeping, not for moment-by-moment computation.

DEVICE	TYPICAL CAPACITY	NOTES
Hard Disk Drive (HDD)	500 GB to several TB	Magnetic coating on rigid spinning platters; read-write heads float just above the surface. Cheap per byte and capacious, but the mechanical seek and rotation make it the slowest common online storage, and it is vulnerable to shock while spinning.
Solid State Drive (SSD)	256 GB to several TB	Flash memory with no moving parts. Far faster than an HDD, silent, and shock-resistant; costs more per byte than an HDD.
CD / CD-ROM	700 MB	Optical disc read by laser. Standard capacity 700 MB.
DVD	4.7 GB (single layer)	Same physical size as a CD but stores about seven times as much, thanks to a finer laser track.
Blu-ray disc	25 GB (single layer)	Blue-violet laser with a still finer track, used for high-definition video and large backups.
USB flash drive	GB to hundreds of GB	Portable flash memory; the everyday medium for carrying files between machines.
Magnetic tape	TB per cartridge	Sequential access only, so it is slow to search but very cheap per byte; still used for large-scale backup and archive.

The practical rule for a working machine is now standard: an SSD for the operating system and active work (for speed), an HDD where bulk capacity at low cost matters, optical or external media and tape-like archives for copies that must survive the failure of the machine itself.

6. Memory units, addresses, and worked conversions

Four bits form one **nibble** — exactly one hexadecimal digit, which is why nibbles appear constantly in low-level work. Eight bits form one byte, two nibbles. A **word** is the natural unit a processor handles at once, commonly 32 or 64 bits on modern machines.

$$1 \text{ nibble} = 4 \text{ bits} \cdot 1 \text{ byte} = 8 \text{ bits} = 2 \text{ nibbles} \cdot 1 \text{ KB} = 1024 \text{ bytes}$$

Every memory location has a unique **address**, a binary number carried on the address bus. The number of addressable locations depends only on the width of that bus:

$$\text{Number of locations} = 2^n, \text{ where } n = \text{number of address lines}$$

WORKED EXAMPLES

Address space: a 16-bit address bus can form 2^{16} = **65,536** distinct addresses, so it can address 65,536 locations. If each location holds one byte, the addressable memory is 65,536 bytes = 65,536 ÷ 1024 = **64 KB**. (Doubling the bus by one line doubles the space: a 20-bit bus addresses 2^{20} = 1,048,576 locations = 1 MB of byte-sized locations.)

Bytes in 3.5 MB: 3.5 MB = 3.5 × 1024 KB = 3584 KB; 3584 KB = 3584 × 1024 bytes = **3,670,016 bytes**. Equivalently in one step: 3.5 × 1024 × 1024 = 3.5 × 1,048,576 = 3,670,016 bytes. Correct.

Addressing is also why memory sizes are so often exact powers of two. A chip with 10 address lines has 2^{10} = 1024 locations; manufacturers build to the binary count, and the familiar 1024-based unit ladder of Topic 1 follows from it.

7. Volatile and non-volatile memory

PROPERTY	VOLATILE MEMORY	NON-VOLATILE MEMORY
Power off	Contents are lost	Contents are retained
Examples	RAM (SRAM and DRAM), cache, registers	ROM family, HDD, SSD, optical discs, flash drives
Speed	Generally the fastest memory in the system	Generally slower (flash EEPROM writes are notably slow)

Role	Working space for running programs	Permanent keeping of programs, data, and firmware

Volatility is the reason for the saving habit every computer user learns: work held only in RAM disappears in a power cut, so documents are written to non-volatile storage regularly. It is also why a machine must boot: at switch-on, RAM is empty, and the non-volatile firmware and disk supply everything that fills it.

Glossary

TERM	MEANING
Memory hierarchy	The layered arrangement of storage from registers down to tape, trading speed against capacity and cost.
Access time	The delay between requesting a memory location and receiving its contents.
Volatile	Describes memory that loses its contents when power is removed, such as RAM.
Cache	A small, very fast memory holding copies of the data and instructions most likely to be needed next.
Locality	The tendency of programs to reuse recent locations (temporal) and nearby locations (spatial).
Nibble	Four bits; exactly one hexadecimal digit.
Firmware	Software stored permanently in ROM or flash, such as the boot program of a machine.
Refresh	The periodic recharging of DRAM cells needed because their capacitors leak charge.
Address bus	The set of lines carrying the binary address of the memory location to be accessed.

Key points & summary

- Memory is a hierarchy because no technology is simultaneously fast, large, and cheap: going down means slower, larger, and cheaper per byte.
- SRAM (flip-flop cells, no refresh, fast, costly) serves as cache; DRAM (capacitor cells, needs refresh, dense, cheap) serves as main memory.

- The ROM family — ROM, PROM, EPROM, EEPROM — is non-volatile and holds firmware; each member is progressively easier to rewrite.
- Cache succeeds because of temporal and spatial locality, reaching hit rates above 90 percent in typical programs.
- Secondary storage is permanent and bulk: HDD for cheap capacity, SSD for speed, CD at 700 MB and DVD at 4.7 GB among optical media.
- Units and addresses are powers of two: 1 nibble = 4 bits; a 16-bit address bus addresses 2^{16} = 65,536 locations = 64 KB; 3.5 MB = 3,670,016 bytes.
- Volatile memory (RAM, cache) is the fast workspace; non-volatile memory (ROM, disks, flash) is the permanent store — a complete system needs both.

Practice

Answer in your own words; brief model answers follow.

1. State the four trends that change as we move down the memory hierarchy, and explain in one sentence why the hierarchy works at all.
2. A processor has a 24-bit address bus and byte-sized locations. How many locations can it address, and how much memory is that in megabytes?
3. How many bytes are there in 2.25 MB? Show the full calculation.
4. Give three differences between SRAM and DRAM, and state where each is normally used and why.
5. Explain both kinds of locality and describe one program behaviour that shows each.
6. A laboratory needs (a) firmware that may be updated twice a year and (b) a cheap archive of ten years of test results. Choose a memory or storage type for each and justify the choice.

Answer key

1. Access time increases, capacity increases, cost per byte decreases, and CPU access frequency decreases. The hierarchy works because programs concentrate their accesses on a small working set at any time (locality), so that set can be kept in the small fast levels.
2. 2^{24} = 16,777,216 locations. At one byte each that is 16,777,216 bytes = 16,777,216 ÷ 1,048,576 = 16 MB.
3. 2.25 × 1024 × 1024 = 2.25 × 1,048,576 = 2,359,296 bytes.
4. SRAM stores a bit in a flip-flop, needs no refresh, is faster, and costs more per byte; DRAM stores a bit as charge on a capacitor, needs periodic refresh, is slower, and is much denser and cheaper. SRAM is used for cache, where small capacity and maximum speed are wanted; DRAM is used for main memory, where gigabytes are needed cheaply.

5. Temporal locality: a recently used location will probably be used again soon — for example a loop variable accessed on every iteration. Spatial locality: locations near a used location will probably be used soon — for example instructions executed in sequence, or array elements processed in order.
6. (a) EEPROM or flash memory, because it can be erased and rewritten electrically in place, unlike one-time PROM or mask ROM. (b) Magnetic tape or hard disk archive (or optical discs for modest volumes), because the results are kept in bulk at the lowest cost per byte and are rarely accessed, so slow sequential access is acceptable.

Texora — course notes prepared from the official syllabus of B.Sc. in Textile Engineering · Made by Sakib